

Regular Expression Syntax (Perl Style)

Regular expression is a pattern of characters which used for describing or matching data according to defined syntax rules. So it can be used to search and replace even the most complicated of string patterns.

The expression must be enclosed in the delimiters, a forward slash (/).

For Example: **Delimiters** can be any non-alphanumeric, non-whitespace ASCII character except the backslash (\) and the null byte. If the delimiter character has to be used in the expression itself, it needs to be escaped by backslash. Since PHP 4.0.4, you can also use Perl-style (), {}, [], and <> matching delimiters.

Ex: /food/

Common Regular Expression Operators

Symbol	DESCRIPTION	EXAMPLES
*	Match zero or more occurrences	"e*" matches a string that has zero or more occurrences of the char "e".
+	Match one or more occurrences.	"e+" matches a string that has one or more occurrences of the char "e".
^	Placed at the beginning of a string, matches that string at the start.	"^Hello" matches any string that starts with the word "Hello".
\$	Placed at the end of a string, matches that string at the end.	"\$goodbye!" matches any string that ends with "goodbye!"
	Used to separate searches.	"a b" matches a string that contains the character "a" or the char "b".
[]	Matches range of characters.	"[a-z]" matches a string that contains a lowercase char from a to z. "[A-D]" matches a string that contains an uppercase char from A to D. "[0-9]" matches a string that contains a numeral from 0 through 9.

		"[1-5]" matches a string that contains a numeral from 1 to 5. [abc] Find any character between the brackets [^abc] Find any character not between the brackets (red blue green) Find any of the alternatives specified
()	Group operators in a sequence.	You can group items together. For example, (5[0-9]+) matches a string that contains a 5 followed by one or more numerals. "5" does not match, while "55" does.
.	Matches any single character.	"[0-9].[0-9]" matches a string that contains a numeral followed by any other character, followed by a numeral.
\	Escape an operator .	You must escape the following operators if you want to search for that particular operator literally: [*\+?{}.]

Quantifiers:

- p+ matches any string containing at least one p.
- p* matches any string containing zero or more p's.
- p? matches any string containing zero or one p.
- p{2} matches any string containing a sequence of two p's.
- p{2,3} matches any string containing a sequence of two or three p's.
- p{2,} matches any string containing a sequence of at least two p's.
- p\$ matches any string with p at the end of it.
- ^p matches any string with p at the beginning of it.
- [^a-zA-Z] matches any string not containing any of the characters ranging from a through z and A through Z.
- p.p matches any string containing p, followed by any character, in turn followed by another p.
- ^.{2}\$ matches any string containing exactly two characters.
- p(hp)* matches any string containing a p followed by zero or more of the sequence hp.
- ?=n matches any string that is followed by a specific string n
Ex: \$str = "Is this all there is"; \$patt1 = "/is(=? all)/"; result = preg_match(\$patt1, \$str);
- ?!n matches any string that is not followed by a specific string n
Ex: \$str = "Is this all there is"; \$patt1 = "/is(?! all)/gi";
- (.*) matches any string enclosed within and (presumably HTML bold tags).

Modifiers

Modifiers are used to perform case-insensitive and global searches:

Modifier	Description
i	Perform a case-insensitive search.
g	Find all occurrences (perform a global search).
m	Treat a string as several (m for multiple) lines. By default, the ^ and \$ characters match at the very start and very end of the string in question. Using the m modifier will allow for ^ and \$ to match at the beginning of any line in a string.
s	Treat a string as a single line, ignoring any newline characters found within; this accomplishes just the opposite of the m modifier.
x	Ignore whitespace and comments within the regular expression.
U	Stop at the first match. Many quantifiers are “greedy”; they match the pattern as many times as possible rather than just stop at the first match. You can cause them to be “ungreedy” with this modifier.

Metacharacters

Metacharacters are characters with a special meaning:

Metacharacter	Description
.	Find a single character, except newline or line terminator
\w	Find a word character
\W	Find a non-word character
\d	Find a digit
\D	Find a non-digit character
\s	Find a whitespace character
\S	Find a non-whitespace character

<code>\b</code>	Find a match at the beginning/end(boundary) of a word
<code>\B</code>	Find a match not at the beginning/end of a word
<code>\0</code>	Find a NUL character
<code>\n</code>	Find a new line character
<code>\f</code>	Find a form feed character
<code>\r</code>	Find a carriage return character
<code>\t</code>	Find a tab character
<code>\v</code>	Find a vertical tab character
<code>\xxx</code>	Find the character specified by an octal number xxx
<code>\xdd</code>	Find the character specified by a hexadecimal number dd
<code>\uxxxx</code>	Find the Unicode character specified by a hexadecimal number xxxx

Examples:

`/wmd/i:` Matches WMD, wMD, WMd, wmd, and any other case variation of the string wmd.

`/taxation/gi:` Case insensitivity locates all occurrences of the word *taxation*. You might use the global modifier to tally up the **total number of occurrences**, or use it in **conjunction** with a **replacement** feature to **replace all occurrences** with some other string.

`/sa\b/` Because the word boundary is defined to be on the right side of the strings, this will match strings like pisa and lisa, but not sand.

`\blinux\b/I` This returns the first case-insensitive occurrence of the word linux.

`/sa\B/` The opposite of the word boundary metacharacter is `\B`, matching on anything but a word boundary. This will match strings like sand and Sally, but not Melissa.

`^\$d+/g` This returns all instances of strings matching a dollar sign followed by one or more digits.

`$line = "Vim is the greatest word processor ever created!";`
`\bVim\b/i` this match if the word *Vim* or *vim* is located, but not *simplevim*, *vims*, or *evim*.

`^[a-zA-Z0-9]{8,10}$/` "it must consist only of alphanumeric char's, and must be 8-10 char's in length (password)

preg_match()

`int preg_match (string pattern, string string [, array matches][, int flags [, int offset]])`

This function searches string for pattern, returning TRUE if it exists and FALSE otherwise. The optional input parameter `pattern_array` can contain various sections of the subpatterns contained in the search pattern, if applicable.

offset can be used to specify the alternate place from which to start the search (in bytes).

The flags parameter:

- **PREG_PATTERN_ORDER** (default) Orders results so that
 `$matches[0]` is an array of all complete pattern matches,
 `$matches[1]` is an array of all strings matched by the first parenthesized subpattern, and so on .
- **PREG_SET_ORDER** orders the array a bit differently than the default setting.
 `$matches [0]` contains elements matched by the first parenthesized subpattern
 `$matches [1]` contains elements matched by the second parenthesized subpattern, and so on.
- **PREG_OFFSET_CAPTURE** If this flag is passed:
 Every occurring match the appendant string offset will also be returned. **Note** that this changes the value of **matches** into an array where every element is an array consisting of the matched string at index **0** and its **string offset** into **subject** at index **1**.

Ex: that uses `preg_match()` to perform a case-sensitive search:

```
<?php
    $line = "Vim is the greatest word processor ever created!";
    if (preg_match("/\bVim\b/i", $line, $match))
        print "Match found!";
?>
```

The above script will confirm a match if the word **Vim** or **vim** is located, but not **simplevim**, **vims**, or **evim**.

A simple pattern can be a single character. A more complicated pattern can consist of more characters

- It can be used for **parsing**, **format checking**, **substitution**, and more.
- It is used to perform powerful pattern-**matching** and **search** and **replace** functions on text.